Day - 5: Interrupts (Part 1)

▼ Interrupts vs Polling | Differences

1. Interrupts

- **Definition:** An interrupt is a hardware signal generated by a peripheral to notify the CPU that it requires a service.
- **Execution Flow:**
  1. The CPU continuously executes the main program.
  2. When a peripheral needs service, it sends an interrupt request.
  3. The CPU temporarily halts the main program and saves its current status and address into the stack (to act as a backup).
  4. The Program Counter (PC) is loaded with the address of the **Interrupt Service Routine (ISR)**, and the CPU jumps to execute the ISR.
  5. Once the ISR is complete, the CPU retrieves (pops) the saved contents from the stack back into the status register and PC, and resumes executing the main program right where it left off.
- **Key Trait:** It is a hardware-based mechanism that can occur at any random time.

2. Polling

- **Definition:** Polling is a software and protocol-based mechanism where the CPU continuously checks the status of peripherals or I/O devices to determine if they need service.
- **Execution Flow:**
  - Instead of waiting for a signal, the processor enters a continuous loop, repeatedly monitoring devices one by one.
  - This checking is done periodically at regular, predefined time intervals (e.g., every 10ms, 100ms, or 1000ms).

- If a device needs service, the processor provides it immediately, then goes right back to monitoring.
- **Example:** Continuously checking a serial communication terminal to see if a new character has been received.

3. Key Differences: Interrupts vs. Polling

Feature	Interrupts	Polling
Initiation	The **peripheral** notifies the CPU that it needs service.	The **CPU** steadily checks the peripherals to see if they need service.
Mechanism Type	Hardware-based mechanism.	Software and protocol-based mechanism.
Service Handler	Handled by a specific Interrupt Handler (ISR).	Handled directly by the processor's main code.
CPU Efficiency	CPU does not waste time; it focuses on the main program and is only "disturbed" when a request occurs.	CPU wastes a massive amount of time continuously looping and monitoring devices.
Occurrence Timing	Can occur randomly at any time.	Happens at strict, predefined intervals.
Handling Multiple Devices	Multiple simultaneous interrupts are served based on an assigned **priority**.	Multiple devices are checked and served sequentially in a **round-robin fashion** (e.g., checking device 1 through 10, then starting back at 1).

▼ Interrupts in Microcontroller | What is an Interrupt

1. What is an Interrupt?

- **Definition:** An interrupt is a signal that causes the processor to halt its current task execution and start running a special program called an Interrupt Service Routine (ISR).
- **Purpose:** It is an essential mechanism in embedded systems used to request immediate service from the processor.

- **Sources:** Interrupts can be generated by various sources, including hardware peripherals (like timers, SPI, or PWM) and software components.

2. The Interrupt Execution Procedure

When an interrupt occurs, the processor follows a specific sequence of steps:

1. **Signal Generation:** A peripheral or software requests an interrupt signal.
2. **Halt Main Program:** The processor pauses the execution of its current main code.
3. **Context Saving:** The processor saves its current state (such as the stack pointer and starting address) by pushing this information into the stack.
4. **Get ISR Address:** The processor looks up the unique address required to service the specific interrupt by checking the **Interrupt Vector Table**.
5. **Execute ISR:** The processor jumps to the retrieved address and executes the Interrupt Service Routine.
6. **Context Restore:** Upon finishing the ISR, the processor retrieves (pops) the previously saved information from the stack.
7. **Resume:** The processor returns to the main program and resumes execution right where it left off.

3. Key Terminology

- **ISR (Interrupt Service Routine):** A small function or piece of code specifically written to handle a particular interrupt.
- **Interrupt Vector Table:** A small allocated memory that stores the unique addresses for every interrupt's corresponding ISR.
- **Interrupt Controller:** A hardware circuit responsible for configuring and managing various interrupts.
- **Context Store & Restore:** The process of automatically saving current task information to the stack when an interrupt occurs (Store), and retrieving it once the ISR finishes (Restore).

4. Types of Interrupts

- **Hardware Interrupts:** Generated by built-in microcontroller peripherals like UART, Timers, PWM, and I2C.
- **Software Interrupts:** Generated by software conditions or errors, such as a "divide by zero" error or exceeding data limits.
- **External vs. Internal:** External interrupts come from the outside world via external pins/switches, whereas internal interrupts are generated by internal hardware.
- **Maskable vs. Non-Maskable:**
  - **Maskable:** Can be temporarily ignored or delayed by the processor.
  - **Non-Maskable:** High-priority interrupts that the processor cannot ignore and must service immediately.
- **Vectored vs. Non-Vectored:**
  - **Vectored:** Every interrupt has a specific address stored in the vector table.
  - **Non-Vectored:** Do not have a specific defined address or rely on a vector table.

5. Priority Schemes

To handle multiple interrupts, the system uses scheduling algorithms to determine which gets serviced first:

- **Fixed Priority Scheme:** Each interrupt is assigned a standard priority level (e.g., 0, 1, 2) that cannot be changed.
- **Dynamic Priority Scheme:** The priority of an interrupt can be changed dynamically based on specific conditions.
- **Priority Masking:** Allows the system to temporarily disable or ignore lower-priority interrupts so it can focus exclusively on higher-priority ones.
- **Round-Robin Scheme:** Multiple interrupts are prioritized and serviced sequentially, one after the other in a continuous loop.

▼ NVIC — Nested Vectored Interrupt Controller

1. Introduction to NVIC

- **Definition:** NVIC stands for **Nested Vectored Interrupt Controller**.
- **Location:** Unlike standard external interrupt controllers, the NVIC is tightly integrated directly into the core ARM Cortex processor.
- **Primary Function:** It is responsible for managing, prioritizing, and executing all interrupts and system exceptions.

2. Sources of Interrupts

The NVIC handles interrupts originating from three major sources:

1. **Core Processor:** Generates internal interrupts known as **exceptions** (which have the highest priority).
2. **Peripherals:** Generates hardware interrupts from microcontroller circuits like Timers, SPI, I2C, CAN, ADC, and external pins.
3. **Software:** Generates software-triggered interrupts.

3. Core Functions of the NVIC

The NVIC is highly advanced and performs several key functionalities:

- **Prioritization:** It assigns a priority level to every interrupt. If multiple interrupts occur simultaneously, the processor executes the highest priority one first.
- **Nested Interrupt Handling & Preemption:** If a higher-priority interrupt is requested while a lower-priority interrupt is currently executing, the NVIC will temporarily suspend (preempt) the lower-priority interrupt to immediately execute the higher-priority one.
- **Interrupt Control:** It controls the enabling and disabling of specific interrupts as needed.
- **Low Latency Processing:** It ensures the processor responds to and processes interrupt requests immediately with minimal delay.
- **Context Switching:** When an interrupt occurs, the NVIC automatically saves the main program's current data (registers, memory locations, and program counter) to the stack as a backup. After the interrupt is handled, it restores this data.

- **System Exceptions Handling:** It manages critical, high-priority exceptions generated directly by the core processor, such as Reset, NMI (Non-Maskable Interrupt), and Hard Faults.
- **Tail Chaining:** If a second interrupt arrives just as the first one is finishing, the NVIC jumps directly to the second interrupt's service routine without wasting time restoring and re-saving the context.

4. Execution Flow: How NVIC Works

When an interrupt is triggered, the NVIC follows a systematic 5-step process:

1. **Interrupt Occurrence:** An interrupt is generated from a peripheral, software, or processor.
2. **Processor State Preservation:** The NVIC performs context switching by saving the processor's current data into the stack.
3. **Vector Table Lookup:** The NVIC looks up the unique memory address associated with the specific interrupt to find its handler.
4. **Handler Execution:** The processor jumps to the assigned address and executes the Interrupt Service Routine (ISR).
5. **Returning from Exception:** Once the ISR is complete, the processor restores its original saved contents from the stack and resumes the main program execution.

5. The Interrupt Vector Table

- **Definition:** The vector table is a specific block of memory where each interrupt is mapped to its corresponding Interrupt Service Routine (ISR) address.
- **Memory Allocation:** Every individual interrupt is allocated exactly **4 bytes** of memory.
- **Table Layout:** The very first address in the vector table (offset `0x00`) is always reserved for the **Stack Pointer**. This is immediately followed by the Reset handler (offset `0x04`) and the NMI handler (offset `0x08`).

6. Exceptions vs. IRQs

The ARM Cortex processor supports a total of **255 interrupts**, which are divided into two categories:

- **Exceptions (16 total):** These are generated internally by the processor core and have the highest priority. They occupy the first 16 slots (numbers 0 to 15) in the vector table. Examples include Reset, NMI, and Bus Fault.
- **IRQs (Interrupt Requests) (240 total):** These are standard interrupts generated by peripherals or software. They range from IRQ0 to IRQ239 (occupying numbers 16 to 255).

▼ Exceptions in ARM Cortex | Reset | NMI | Hard Fault | Usage Fault

1. Exceptions vs. Interrupts

While they follow the same execution procedure, there is a distinct difference in where they originate:

- **Exceptions:** These are generated directly from the **processor core** side. An exception occurs when the processor encounters a software error or fault situation during code execution, such as dividing by zero, undefined instructions, or invalid memory access.
- **Interrupts:** These are generated from the **microcontroller peripheral** side. They are hardware requests from components like Timers, ADC, SPI, PWM, or I2C.
- **Management:** Both exceptions and interrupts are handled by the processor's **NVIC** (Nested Vector Interrupt Controller).

2. The Exception Handling Execution Flow

When an exception occurs, the processor handles it systematically by switching to a privileged mode and executing an exception handler. The step-by-step process is:

1. **Halt Main Task:** The current execution of the main code is temporarily suspended.
2. **Context Storing (Stacking):** The processor saves key state information (such as the Program Counter, memory addresses, and specific registers)

into the stack.

3. **Fetch Handler Address:** The processor looks up the appropriate exception handler address from the interrupt vector table.
4. **Execute Handler:** The processor enters handler mode and begins executing the specific handler function.
5. **Context Restoring (Unstacking):** Upon completion, the processor retrieves the previously saved context from the stack, returns to the main code, and resumes execution.

3. Types of ARM Cortex Exceptions

The ARM Cortex architecture defines multiple built-in system exceptions. Here are the primary ones:

- **Reset:** The highest priority exception. It occurs during a soft reset or power-on reset.
- **NMI (Non-Maskable Interrupt):** The second-highest priority exception. It is a critical interrupt that cannot be masked, ignored, or preempted by other interrupts.
- **Hard Fault:** A general system exception that triggers when an error cannot be managed by any of the other specific fault handlers.
- **Memory Management Fault:** Occurs when attempting to access reserved data or protected memory managed by the Memory Protection Unit.
- **Bus Fault:** Occurs when trying to fetch an instruction or access data from a protected or reserved memory space.
- **Usage Fault:** Triggered by software execution errors, specifically scenarios like a "divide by zero", undefined instructions, or illegal/unaligned memory access.
- **Secured Fault:** Occurs when attempting to access data from a strictly secured memory area or interacting with a secured algorithm.
- **SVC (Supervisory Call):** A system call exception primarily used in an Operating System (RTOS) environment to allow application tasks to run in a service mode.

- **Debug Monitor:** Triggered when the processor is in debugging mode and encounters an issue with a breakpoint or watch window.
- **PendSV (Pendable Service Call):** Exclusively used in an OS environment to handle "context switching" when the system needs to switch rapidly between two or more different running tasks.
- **SysTick Exception:** Generated internally by the processor's built-in SysTick timer whenever the timer tick expires (times out) or overruns.